Library Database Mini Project

CMPT 354

Karan Passi - 301609118

Tushar Singh - 301560473

Step 1: Project Specifications

For our project, we made a database and a small Python application for a library. We wanted it to cover the main things that happen in a library, such as keeping track of items, letting members borrow and return them, holding events, and keeping records of staff. We also included donations, volunteer applications, and help requests. We designed the project for one library location and did not model multiple branches.

Library Items and Copies

The library has different kinds of items, including print books, online books, magazines, scientific journals, and records. For each item, we keep its title and the name of its creator if one is available. An item can be in the collection, removed from the collection, or marked as prospective if the library may add it in the future. Prospective items cannot be borrowed.

We decided to separate an item from its physical copies. For example, the library might have three copies of the same print book, so they should not have to be entered as three completely different books. Each physical copy has a copy number and a condition. The conditions we use are usable, damaged, under repair, lost, and retired. Only a usable copy of an item that is currently in the collection can be borrowed.

Online books are also kept in the catalogue, but they have an access link instead of a physical copy. We do not track digital licences or concurrent access limits.

Members, Borrowing, and Fines

The library keeps information about its members, including their name, contact information, and membership status. A member can be active, suspended, or expired. We still keep suspended and expired members because they may have older records connected to them, but only an active member is allowed to start a new loan.

When someone borrows an item, we record the member, the exact copy they borrowed, the borrow date, and the due date. Once they return it, the return date is added to the same loan. We do not delete the loan after the return.

A copy can only have one active loan at a time. If an item is returned late, damaged, or lost, the member may receive a fine. We record the type of fine, its amount, the date it was given, and whether it is unpaid, paid, or waived.

Staff and Library Events

We keep personnel records for library staff as required in the assignment. This includes the staff member's name, job title, email, hire date, department, and employment status. The employment status can be active, on leave, or former.

The library can hold events such as book clubs, author talks, workshops, art shows, and film screenings. Events are free. Each event is meant for a specific audience, such as children,

teenagers, adults, or families. We record the title, category, description, target audience, time, status, and registration limit. An event can be scheduled, cancelled, or completed.

Each event is held in one library room and is organised by one staff member. A room has a room number and a capacity. The event registration limit cannot be greater than the room capacity, and two events cannot be scheduled in the same room at overlapping times.

Members can register for a scheduled event if there is still space. Each member and event pair has one registration record. Its status can be registered, cancelled, attended, or no-show, so separate registration attempts are not stored.

Other Library Services

Members can offer physical items as donations. We keep the member who made the offer, the date, a description of the item, and whether the offer is pending, accepted, or rejected. If the library accepts it and adds it to the collection, the donation can later be connected to the copy created from it.

A member can apply to volunteer for a specific event or for general library work. We record the date of the application and its status: pending, approved, rejected, or withdrawn.

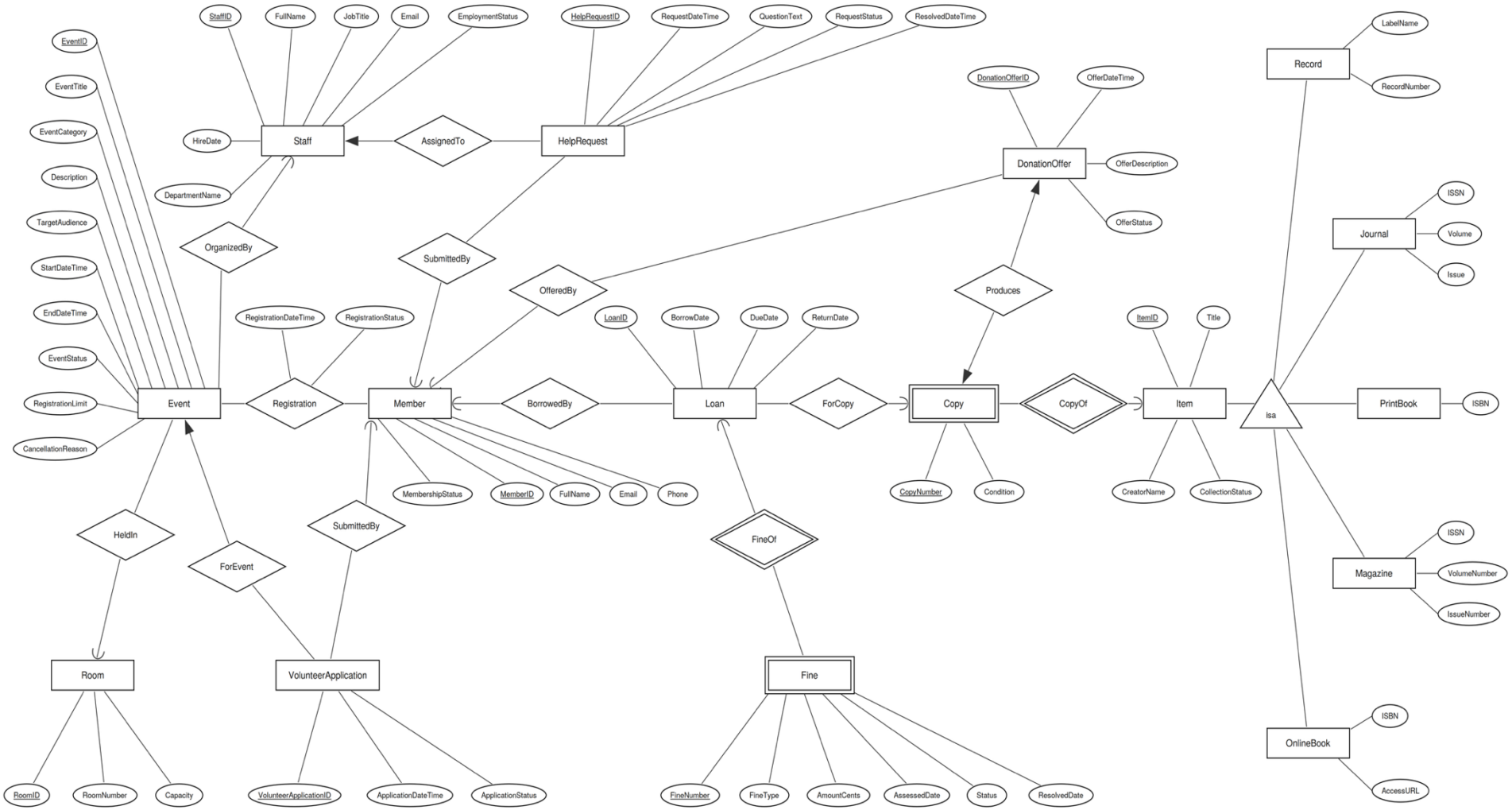
Members can also send questions or help requests to a librarian. We record the date, the question, the assigned staff member if there is one, and whether the request is pending, in progress, or resolved. A request can later be updated when someone starts working on it or resolves it.

Application Requirements

The application lets a library user perform the eight actions required in the assignment:

1. Find an item in the library.
2. Borrow an available item.
3. Return a borrowed item.
4. Donate an item to the library.
5. Find a library event.
6. Register for an event.
7. Volunteer for the library.
8. Ask a librarian for help.

Step 2: ER Diagram



Step 3: BCNF Analysis

We turned the ER diagram into relational tables, including the regular entities, weak entities, and the Registration relationship with its attributes. For each table, we looked at its candidate keys and checked the non-trivial functional dependencies using the BCNF rule.

Result: All Relations Satisfy BCNF

Simple entity tables – These tables are Staff, Event, Member, Room, HelpRequest, VolunteerApplication, DonationOffer, Item, and Loan. Each table has a single-attribute primary key, such as StaffID or EventID, that determines the other attributes in that table. For example:

StaffID → FullName, JobTitle, Email, EmploymentStatus, HireDate, DepartmentName

Some tables also have attributes declared as UNIQUE. Staff.Email and Room.RoomNumber are examples. These attributes can also identify a row, so they are alternate keys and do not violate BCNF.

The other main functional dependencies are:

MemberID → FullName, Email, Phone, MembershipStatus

RoomID → RoomNumber, Capacity

EventID → EventTitle, EventCategory, Description, TargetAudience, StartDateTime, EndDateTime, EventStatus, RegistrationLimit, CancellationReason, RoomID, OrganizerStaffID

ItemID → Title, CreatorName, CollectionStatus

LoanID → MemberID, ItemID, CopyNumber, BorrowDate, DueDate, ReturnDate

DonationOfferID → MemberID, OfferDateTime, OfferDescription, OfferStatus, ProducedItemID, ProducedCopyNumber

VolunteerApplicationID → MemberID, EventID, ApplicationDateTime, ApplicationStatus

HelpRequestID → MemberID, StaffID, RequestDateTime, QuestionText, RequestStatus, ResolvedDateTime

Weak entities – Copy has the key (ItemID, CopyNumber), while Fine has the key (LoanID, FineNumber). CopyNumber and FineNumber are not globally unique. They only distinguish copies belonging to the same item or fines belonging to the same loan. The functional dependencies are:

(ItemID, CopyNumber) → Condition

(LoanID, FineNumber) → FineType, AmountCents, AssessedDate, Status, ResolvedDate

In both tables, the complete composite key is needed to determine the remaining attributes. Therefore, Copy and Fine satisfy BCNF.

Registration relationship – Registration has the key (MemberID, EventID). The registration date and status belong to one specific member and event pair. They do not depend on MemberID or EventID by itself.

(MemberID, EventID) → RegistrationDateTime, RegistrationStatus

The complete primary key determines the other attributes, so Registration satisfies BCNF.

ISA hierarchy – Item is the main table, while PrintBook, Journal, Magazine, Record, and OnlineBook are the item subtypes. Each subtype uses the ItemID inherited from Item as its primary key. The attributes belonging to each subtype, such as ISBN or the combination of ISSN, volume, and issue, depend on ItemID.

Some subtype tables also have values or combinations declared as UNIQUE. These can act as alternate keys, so they are superkeys and do not violate BCNF.

Relationships implemented using foreign keys – AssignedTo, OrganizedBy, SubmittedBy, HeldIn, ForEvent, BorrowedBy, ForCopy, FineOf, CopyOf, and OfferedBy are stored using foreign keys in the appropriate tables. For example, Event stores OrganizerStaffID, while HelpRequest stores MemberID and StaffID. These foreign keys connect rows in different tables, but they do not determine the remaining attributes in their tables. Therefore, they do not create BCNF violations.

Produces is an optional one-to-one relationship. DonationOffer stores the nullable foreign key (ProducedItemID, ProducedCopyNumber) when a donation creates a copy. The pair is declared as UNIQUE so that the same copy cannot be connected to more than one donation offer.

Conclusion

For every non-trivial functional dependency in our design, the determinant is a superkey. Therefore, every table satisfies BCNF, and no further BCNF decomposition is needed.